

Wenshu Zhao

(+1) 626-997-6248 | wszhao87@gmail.com | <https://www.linkedin.com/in/wenshu-zhao> | <https://gin-87.github.io/>

EDUCATION

University of Washington, Seattle	Sep.2024 - Jun.2026(Expected)
M.S. Information Science (Specialization: Data Science & AI)	GPA: 3.9/4.0
China Pharmaceutical University, Nanjing	Sep.2020 - Jun.2024
B. S. Information Management and Information Systems	GPA: 3.7/4.0

SKILLS

Programming Languages: Java, Python, JavaScript, TypeScript, SQL, C++, Kotlin, Shell, HTML/CSS
Frameworks & Tools: Spring Boot, React, Vue.js, FastAPI, LangChain, LangGraph, Hadoop, Apache Spark, Maven, Expo
AI/ML: PyTorch, TensorFlow, OpenAI API, Hugging Face Transformers, RAG, PEFT (LoRA)
Databases & Cloud: PostgreSQL, MySQL, MongoDB, Redis, AWS, Azure, GCP, Docker, Kubernetes, Nginx, Kafka, Linux
Concepts: Distributed Systems, Microservices, System Design, CI/CD, Data Structures & Algorithms, SDLC, A/B testing

PROFESSIONAL EXPERIENCE

Software Engineer (Capstone)

Voraa

Sep.2025 - Present

- Shipped Deal-Breaker Mode with **Flutter** and **Firebase Cloud Functions (Node.js)** on **GCP**, implementing multi-constraint filtering across 10 dimensions on 300+ venue; improved venue health score from 5.7 to 7.8.
- Designed a 5-axis weighted Match Score model as a domain-only layer, with deterministic test injection and a filter → score → sort → paginate pipeline processing 200 venues under 2ms.
- Built a **Relax Suggestion Engine** detecting thin-result queries and surfacing ranked constraint relaxations, increasing venue visits/week from 1.18 to 1.52 and reducing never-ramped venues from 59% to 40%.
- Designed a user behavior analytics pipeline with **GCP Pub/Sub** and **BigQuery** capturing ~500 behavioral events/day; added per-user rate limiting and validation with structured **Cloud Logging** for observability.
- Integrated **Claude Code** into the development workflow for architecture decisions and scaffolding; enforced 100% **Jest** coverage on scoring module via **Husky + lint-staged** pre-commit automation, reducing iteration cycles by ~30%.

Software Engineer Intern

IpserLab

Jun.2025 - Dec.2025

- Architected a schema-driven form engine in **React Native** and **TypeScript** with custom hooks, and server-side JSON Schema validation, reducing new-field addition cost across 6-step workflows with 28+ dynamic fields.
- Built **RESTful** backend services in **Java 21** using a Command-pattern router and **PostgreSQL**-backed multipart ingestion; migrated storage from local disk to **AWS S3** with presigned URLs, improving scalability and access control.

Software Engineer Intern

Legend Biotech

Feb.2023 - Apr.2023

- Engineered patient data management modules for a CAR-T treatment tracking system using **Java(Spring Boot)** and **JavaScript**, enabling real-time treatment schedule updates and progress tracking for 200+ patients.

PROJECT EXPERIENCE

Seattle Source Ranker

Sep.2025 - Dec.2025

- Built a distributed ingestion system with **Celery & Redis** to collect and validate **400K+** repositories from **28K** Seattle developers, achieving **8.1×** higher throughput and **99.98%** reliability despite API rate limits.
- Architected a fault-tolerant API layer combining **GraphQL** and **REST** fallback strategies to correct inconsistent watcher metrics, eliminating 8K invalid repositories and improving ranking data accuracy.
- Implemented a rate-limit-aware token rotation system with adaptive backoff and idempotent retries, reducing rate-limit failures by **92%** and enabling full dataset rebuilds within 60-90 minutes.
- Deployed **Prometheus + Grafana** to monitor queue backlog and pipeline health, eliminating observability blind spots.
- Automated daily updates and deployment via **GitHub Actions**, enabling real-time search and sub-50ms load times.

AI Letter Generation Platform

Jun.2025 - Sep.2025

- Built a **FastAPI** service integrating GPT and Doubao for LLM-powered letter generation with role-based prompts, multilingual support, and multi-turn context; added a **LangChain + ChromaDB RAG** pipeline for grounded responses.
- Implemented **SSE-based** streaming output and exposed an async task status API over **RabbitMQ** queues, eliminating client-side long-polling under high LLM latency.
- Added per-user token tracking via **Redis** with configurable rate limits and instrumented the full Java → RabbitMQ → Python → LLM path with **OpenTelemetry**, exposing cross-service latency breakdowns.
- Containerized Java + Python services with **Docker** and deployed to **Kubernetes** on AWS with HPA auto-scaling; used **Nginx** ingress, **MongoDB** for content storage, and **CloudWatch** alarms for proactive LLM failure alerting.